

Q1)

1. Design:

```
1  module queue();
2
3      int j;
4      int q[$];
5      int temp;
6
7      initial begin
8          j = 1;
9          q = {0, 2, 5};
10         q.insert(1, j);
11         $write("Q Queue:{"");
12         foreach(q[i])begin
13             $write("
14 , ", q[i]);
15         $display("}\n");
16
17         q.delete(1);
18         $write("Q Queue after deleting index 1:{"");
19         foreach(q[i])begin
20             $write("
21 , ", q[i]);
22         $display("}\n");
23
24         q.push_front(7);
25         $write("Q Queue pushing 7 in front:{"");
26         foreach(q[i])begin
27             $write("
28 , ", q[i]);
29         $display("}\n");
30
31         q.push_back(9);
32         $write("Q Queue after pushing 9 in back:{"");
33         foreach(q[i])begin
34             $write("
35 , ", q[i]);
36         $display("}\n");
37
38         j = q.pop_back();
39         $display("J =
40 ", j); $write("Q Queue:{"");
41         foreach(q[i])begin
42             $write("
43 , ", q[i]);
44         $display("}\n");
```

```

46     j = q.pop_front();
47     $display("j = %0d", j);
48     $write("Q Queue:{"");
49     foreach(q[i])begin
50         $write("%0d, ", q[i]);
51     end
52     $display("}\n");|
53
54     q.reverse();
55     $write("Q Queue Reversed:{"");
56     foreach(q[i])begin
57         $write("%0d, ", q[i]);
58     end
59     $display("}\n");
60
61     q.sort();
62     $write("Q Queue Sorted:{"");
63     foreach(q[i])begin
64         $write("%0d, ", q[i]);
65     end
66     $display("}\n");
67
68     q.rsort();
69     $write("Q Queue Reversed Sorted:{"");
70     foreach(q[i])begin
71         $write("%0d, ", q[i]);
72     end
73     $display("}\n");
74
75     q.shuffle();
76     $write("Q Queue shuffled:{"");
77     foreach(q[i])begin
78         $write("%0d, ", q[i]);
79     end
80     $display("}\n");
81
82     end
83
84 endmodule

```

2. Questasim Output:

```
vsim(pause)/> run -all
# Q Queue:{0, 1, 2, 5, }
#
# Q Queue after deleting index 1:{0, 2, 5, }
#
# Q Queue pushing 7 in front:{7, 0, 2, 5, }
#
# Q Queue after pushing 9 in back:{7, 0, 2, 5, 9, }
#
# J = 9
# Q Queue:{7, 0, 2, 5, }
#
# J = 7
# Q Queue:{0, 2, 5, }
#
# Q Queue Reversed:{5, 2, 0, }
#
# Q Queue Sorted:{0, 2, 5, }
#
# Q Queue Reversed Sorted:{5, 2, 0, }
#
# Q Queue shuffled:{2, 0, 5, }
#
```

Q2)

1. Design:

```
Digital_Verification_Diploma > Session_3 > Extra_Assignment > Q2 > adder.v > adder
1  module adder (
2      input  clk,
3      input  reset,
4      input  signed [3:0] A, // Input data A in 2's complement
5      input  signed [3:0] B, // Input data B in 2's complement
6      output reg signed [4:0] C // Adder output in 2's complement
7      );
8
9      // Register output C
10     always @(posedge clk or posedge reset) begin
11         if (reset)
12             C <= 5'b0;
13         else
14             C <= A + B;
15     end
16 endmodule
17
```

2.

3. Testbench Code

Test Bench:

```
1  import adder_pkg::*;
2
3  module adder_tb();
4
5      localparam MAXPOS = 7;
6      localparam MAXNEG = -8;
7
8      logic CLK;
9      logic reset;
10     logic signed [3:0] A; // Input data A in 2's complement
11     logic signed [3:0] B; // Input data B in 2's complement
12     logic signed [4:0] C_dut; // Adder output in 2's complement
13
14     int error_count, correct_count;
15
16     adder DUT (CLK, reset, A, B, C_dut);
17
18     Adder_inputs inputs = new;
19
20     initial begin
21         CLK = 0;
22         forever
23             #1 CLK = ~CLK;
24     end
25
26     task reset_assert();
27         reset = 1;
28         check_result(0);
29         reset = 0;
30     endtask
31
32     task check_result(input logic signed [4:0] C_expected);
33         @(negedge CLK);
34         $display("Expected:%d | DUT: %d", C_expected, C_dut);
35         if(C_dut != C_expected) begin
36             error_count++;
37             $stop;
38         end
39         else begin
40             correct_count++;
41         end
42     endtask
43
44     always @(posedge CLK) begin
45         inputs.sample();
46     end
47
48
49     initial begin
50         A = 0;
51         B = 0;
52         correct_count = 0;
53         error_count = 0;
54
55         repeat(1000)begin
56             assert(inputs.randomize());
57             reset = inputs.reset;
58             A = inputs.A;
59             B = inputs.B;
60             @(negedge CLK);
61         end
62     end
63 endmodule
```

Package:

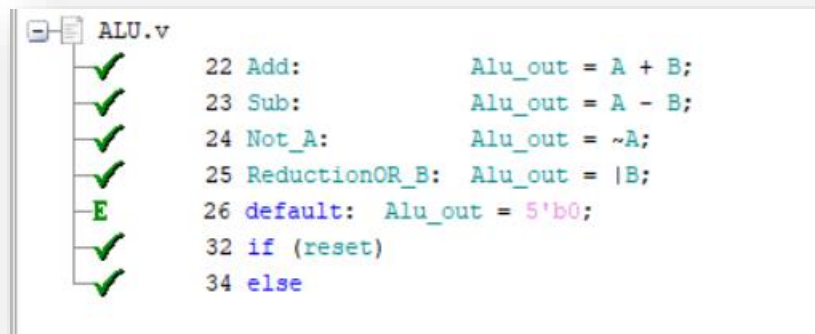
```
1 package adder_pkg;
2
3     typedef enum {MAXNEG = -8, ZERO = 0, MAXPOS = 7} CORNERS_e;
4
5     class Adder_inputs;
6
7         rand logic clk;
8         rand logic reset;
9         rand logic signed [3:0] A; // Input data A in 2's complement
10        rand logic signed [3:0] B; // Input data B in 2's complement
11
12        constraint c_reset{
13            reset dist {0:/90, 1:/10};
14        }
15
16        constraint c_adder_inputs{
17            A dist {MAXNEG:/30, ZERO:/30, MAXPOS:/30, [-7:6]:/10};
18            B dist {MAXNEG:/30, ZERO:/30, MAXPOS:/30, [-7:6]:/10};
19        }
20
21        covergroup Covgrp_A;
22
23            Val_A: coverpoint A{
24                bins data_0 = {ZERO};
25                bins data_max = {MAXPOS};
26                bins data_min = {MAXNEG};
27                bins data_default = default;
28            }
29
30            TRANS_A: coverpoint A {
31                bins data_0max = (0 => MAXPOS);
32                bins data_maxmin = (MAXPOS => MAXNEG);
33                bins data_minmax = (MAXNEG => MAXPOS);
34            }
35
36        endgroup
37
38        covergroup Covgrp_B;
39
40            Val_B: coverpoint B{
41                bins data_0 = {ZERO};
42                bins data_max = {MAXPOS};
43                bins data_min = {MAXNEG};
44                bins data_default = default;
45            }
46
47            TRANS_B: coverpoint B {
48                bins data_0max = (0 => MAXPOS);
49                bins data_maxmin = (MAXPOS => MAXNEG);
50                bins data_minmax = (MAXNEG => MAXPOS);
51            }
52
53        endgroup
54
55        function new();
56            Covgrp_A = new;
57            Covgrp_B = new;
58        endfunction
59
60        task sample();
61            if(~reset)begin
62                Covgrp_A.sample();
63                Covgrp_B.sample();
64            end
65        endtask
66
67    endclass
68
69 endpackage
```

4. Do File:

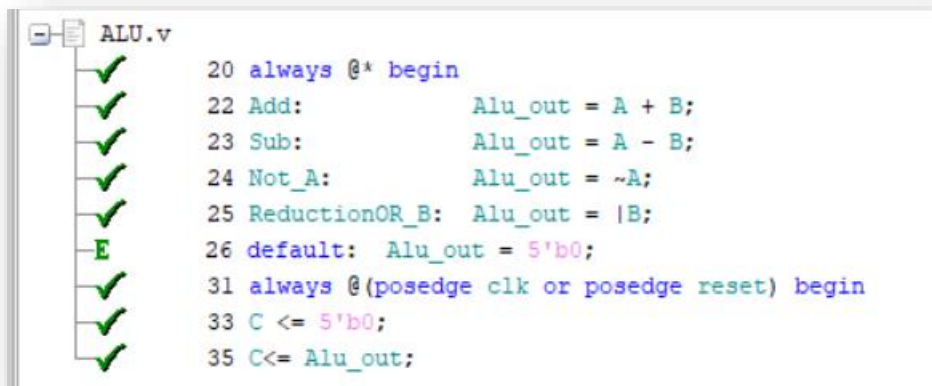
```
1 vlib work
2 vlog adder_pkg.sv adder.v adder_tb.sv +cover -covercells
3 vsim -voptargs=+acc work.adder_tb -cover
4 add wave *
5 coverage save adder_tb.ucdb -onexit -du work.adder
6 run -all
7 #quit -sim
8 #vcover report adder_tb.ucdb -details -annotate -all -output coverage_rpt.txt
```

5. Code Coverage:

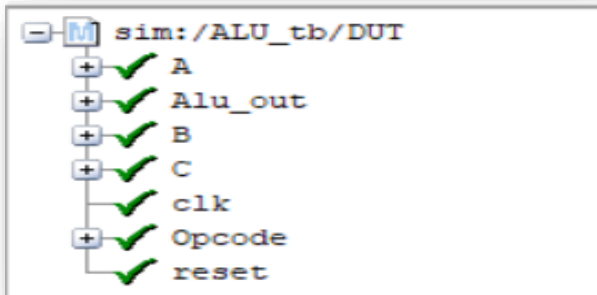
Branch Coverage:



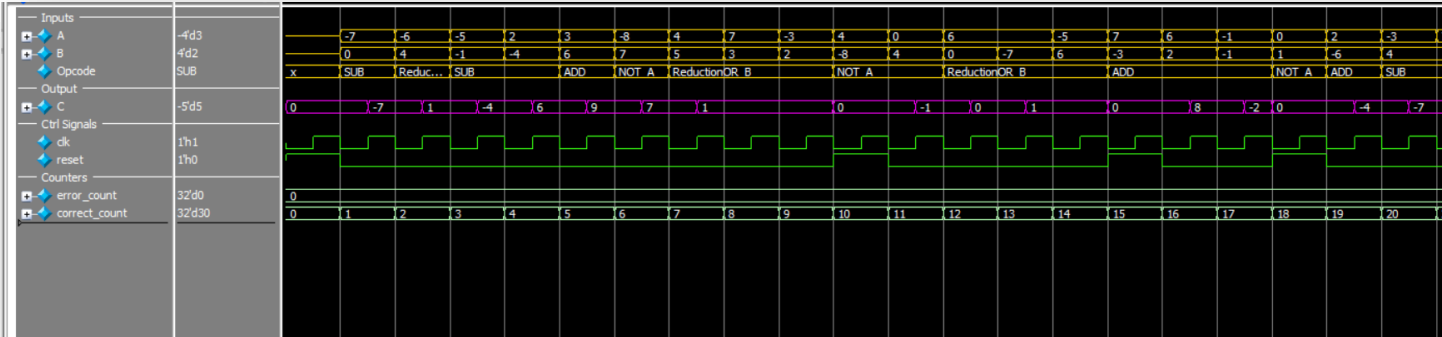
Statement Coverage:



Toggle Coverage:



6. Questasim Waveform



7. Coverage Report:

Branch Coverage:

```
=====
=== Instance: /\ALU_tb#DUT
=== Design Unit: work.ALU_4_bit
=====

Branch Coverage:
  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Branches              6      6      0    100.00%

=====Branch Details=====

Branch Coverage for instance /\ALU_tb#DUT

  Line    Item          Count    Source
  ----    -
  File ALU.v
  -----CASE Branch-----
  21              30    Count coming in to CASE
  22      1              8      Add:      Alu_out = A + B;
  23      1              8      Sub:      Alu_out = A - B;
  24      1              5      Not_A:    Alu_out = ~A;
  25      1              9      ReductionOR_B: Alu_out = |B;

Branch totals: 4 hits of 4 branches = 100.00%
```


Statement Coverage:

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	8	8	0	100.00%

=====Statement Details=====

Toggle & Total Coverage:

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	44	44	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\ALU_tb#DUT --

Node	1H->0L	0L->1H	"Coverage"
A[0-3]	1	1	100.00
Alu_out[4-0]	1	1	100.00
B[0-3]	1	1	100.00
C[4-0]	1	1	100.00
Opcode[0-1]	1	1	100.00
clk	1	1	100.00
reset	1	1	100.00

Total Node Count = 22
Toggled Node Count = 22
Untoggled Node Count = 0

Toggle Coverage = 100.00% (44 of 44 bins)

Total Coverage By Instance (filtered view): 100.00%

8. Functional Coverage:

```

Covergroup instance \adder_pkg::Adder_inputs::Covgrp_A
    100.00%      100      -      Covered
covered/total bins:      6      6      -
missing/total bins:      0      6      -
% Hit:      100.00%      100      -
Coverpoint Val_A      100.00%      100      -      Covered
    covered/total bins:      3      3      -
    missing/total bins:      0      3      -
    % Hit:      100.00%      100      -
    bin data_0      273      1      -      Covered
    bin data_max      280      1      -      Covered
    bin data_min      263      1      -      Covered
    default bin data_default      87      -      Occurred
Coverpoint TRANS_A      100.00%      100      -      Covered
    covered/total bins:      3      3      -
    missing/total bins:      0      3      -
    % Hit:      100.00%      100      -
    bin data_0max      88      1      -      Covered
    bin data_maxmin      87      1      -      Covered
    bin data_minmax      80      1      -      Covered
TYPE /adder_pkg/Adder_inputs/Covgrp_B      100.00%      100      -      Covered
    covered/total bins:      6      6      -
    missing/total bins:      0      6      -
    % Hit:      100.00%      100      -

```

```

TYPE /adder_pkg/Adder_inputs/Covgrp_B      100.00%      100      -      Covered
    covered/total bins:      6      6      -
    missing/total bins:      0      6      -
    % Hit:      100.00%      100      -
Coverpoint Val_B      100.00%      100      -      Covered
    covered/total bins:      3      3      -
    missing/total bins:      0      3      -
    % Hit:      100.00%      100      -
Coverpoint TRANS_B      100.00%      100      -      Covered
    covered/total bins:      3      3      -
    missing/total bins:      0      3      -
    % Hit:      100.00%      100      -
Covergroup instance \adder_pkg::Adder_inputs::Covgrp_B
    100.00%      100      -      Covered
covered/total bins:      6      6      -
missing/total bins:      0      6      -
% Hit:      100.00%      100      -
Coverpoint Val_B      100.00%      100      -      Covered
    covered/total bins:      3      3      -
    missing/total bins:      0      3      -
    % Hit:      100.00%      100      -
    bin data_0      259      1      -      Covered
    bin data_max      282      1      -      Covered
    bin data_min      269      1      -      Covered
    default bin data_default      93      -      Occurred
Coverpoint TRANS_B      100.00%      100      -      Covered
    covered/total bins:      3      3      -
    missing/total bins:      0      3      -
    % Hit:      100.00%      100      -
    bin data_0max      77      1      -      Covered
    bin data_maxmin      80      1      -      Covered
    bin data_minmax      85      1      -      Covered

```

TOTAL COVERGROUP COVERAGE: 100.00% COVERGROUP TYPES: 2

Total Coverage By Instance (filtered view): 100.00%

Q3)

1. Design

```
 8 module FSM_010(clk, rst, x, y, users_count);
 9     parameter IDLE = 2'b00;
10     parameter ZERO = 2'b01;
11     parameter ONE  = 2'b10;
12     parameter STORE = 2'b11;
13
14     input clk, rst, x;
15     output y;
16     output reg [9:0] users_count;
17
18     reg [1:0] cs, ns;
19
20     always @(*) begin
21         case (cs)
22             IDLE:
23                 if (x)
24                     ns = IDLE;
25                 else
26                     ns = ZERO;
27             ZERO:
28                 if (x)
29                     ns = ONE;
30                 else
31                     ns = ZERO;
32             ONE:
33                 if (x)
34                     ns = IDLE;
35                 else
36                     ns = STORE;
37             STORE:
38                 if (x)
39                     ns = IDLE;
40                 else
41                     ns = ZERO;
42             default: ns = IDLE;
43         endcase
44     end
45
46     always @(posedge clk or posedge rst) begin
47         if(rst) begin
48             cs <= IDLE;
49         end
50         else begin
51             cs <= ns;
52         end
53     end
54
55     always @(posedge clk or posedge rst) begin
56         if(rst) begin
57             users_count <= 0;
58         end
59         else begin
60             if (cs == STORE)
61                 users_count <= users_count + 1;
62             end
63         end
64
65         assign y = (cs == STORE)? 1:0;
66
67     endmodule
```

2. Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FSM_1	Checks output when reset is asserted	Directed at the beginning and then with the randomized test with constraints as off most of the time	-	A checker to make sure output is correct
FSM_2	Checks output with various input combinations using randomized inputs	Directed during simulation	Covers X_trans	A checker to make sure output is correct

4. Test Bench:

Package:

```
1  package FSM_010_pkg;
2
3      typedef enum logic [1:0] {IDLE, ZERO, ONE, STORE} state_e;
4
5      class fsm_transaction;
6
7          rand logic      x, rst, y_exp;
8          rand logic [9:0] user_count_exp;
9          bit clk;
10
11         constraint c_x{
12             x dist{0:/67, 1:/33};
13         }
14
15         constraint c_rst{
16             rst dist{0:/95, 1:/5};
17         }
18
19         covergroup cg_x @(posedge clk);
20             x_trans: coverpoint x{
21                 bins x_0_1_0 = (0 => 1 => 0);
22             }
23         endgroup
24
25         function new(logic x = 0, rst = 0, y = 0, logic [9:0] user_count_exp = 0);
26
27             this.x = x;
28             this.rst = rst;
29             this.y_exp = y;
30             this.user_count_exp = user_count_exp;
31
32             cg_x = new;
33
34         endfunction
35     endclass
36
37 endpackage
```

Test Bench:

```
1 import FSM_010_pkg::*;
2
3 module FSM_010_tb();
4
5     logic clk, rst, x;
6     logic y_dut, y_golden;
7     logic [9:0] users_count_dut, users_count_golden;
8
9     fsm_transaction inputs = new;
10
11     int error_count, correct_count;
12
13     FSM_010 dut (.x(x), .clk(clk), .rst(rst), .y(y_dut), .users_count(users_count_dut));
14     Sequence_detect Golden (.x(x), .clk(clk), .rst(rst), .y(y_golden), .count(users_count_golden));
15
16     initial begin
17         clk = 0;
18         forever
19             #1 clk = ~clk;
20     end
21
22     initial begin
23         error_count = 0; correct_count = 0;
24         //FSM_1
25         reset_assert();
26
27         //FSM_2
28         repeat(30)begin
29             assert(inputs.randomize());
30             rst = inputs.rst;
31             x = inputs.x;
32             check_result();
33         end
34
35         //Extra Tests for 100% coverage
36         rst = 0; x = 0; check_result(); //0
37         rst = 0; x = 1; check_result(); //10
38         rst = 0; x = 0; check_result(); //010
39         rst = 0; x = 1; check_result(); //1010
40
41         $display("Total Tests:
42 | Correct: %0d;
43 | Error: %0d;
44 ", correct_count + error_count, correct_count, error_count);
45         task reset_assert();
46             rst = 1;
47             check_result;
48             rst = ~rst;
49         endtask
50
51         task check_result();
52             @(negedge clk);
53             if(y_dut != y_golden || users_count_dut != users_count_golden) begin
54                 error_count++;
55                 $display("Expected: Y:
56 , Count: %0d;
57 | DUT: %0d
58 , Count: %0d;
59 <---Error", $display(0); Expected: %0d; DUT: %0d;
60 , Count: %0d;
61 | DUT: %0d
62 , Count: %0d;
63 ", y_golden, users_count_golden, y_dut, users_count_dut);
64 endmodule
```

Golden Model:

```
1  module Sequence_detect #(
2      parameter IDLE  = 2'b00,
3      parameter ZERO  = 2'b01,
4      parameter ONE   = 2'b10,
5      parameter STORE = 2'b11
6  ) (
7      input          x,
8      input          clk,
9      input          rst, //Active High Asynch
10     output reg      y,
11     output reg [9:0] count
12 );
13
14 reg [1:0] state, next_state;
15
16 //current state logic
17 always @(posedge clk or posedge rst) begin
18     if(rst) begin
19         state <= IDLE;
20     end
21     else
22         state <= next_state;
23 end
24
25 //next state logic
26 always @(*) begin
27     case (state)
28     IDLE:
29         if(x == 0)
30             next_state = ZERO;
31         else
32             next_state = IDLE;
33     ZERO:
34         if(x == 1)
35             next_state = ONE;
36         else
37             next_state = ZERO;
38     ONE:
39         if(x == 0)
40             next_state = STORE;
41         else
42             next_state = IDLE;
43     STORE:
44         if(x == 0)
45             next_state = ZERO;
46         else
47             next_state = IDLE;
48     default: next_state = IDLE;
49     endcase
50
51 end
52
53 //output logic
54 always @(*)begin
55     y = (state == STORE) ? 1'b1 : 1'b0;
56 end
57
58 always @(posedge clk)begin
59     if(rst)
60         count <= 0;
61     else if(state == STORE)
62         count <= count + 1;
63 end
64
65 endmodule
```

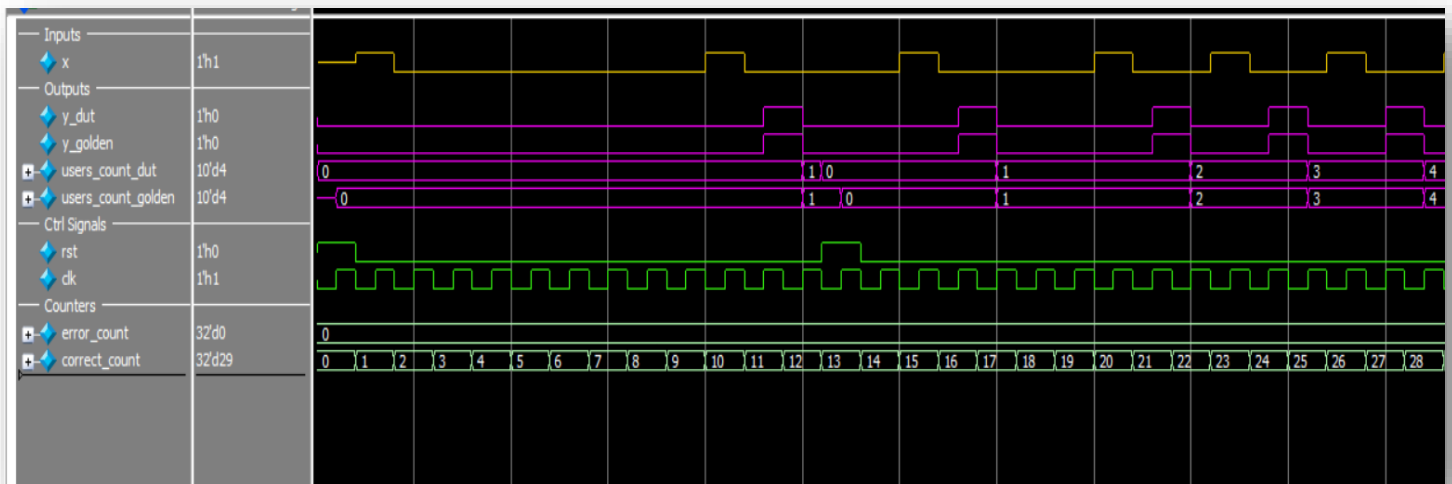
5. Do File

```

1  vlib work
2  vlog FSM_010.v FSM_010_Golden.v FSM_010_pkg.sv FSM_010_tb.sv +cover -covercells
3  vsim -voptargs=+acc work.FSM_010_tb -cover
4  do wave.do
5  coverage save FSM_010_tb.ucdb -onexit -du work.FSM_010
6  run -all
7  coverage exclude -du FSM_010 -togglenode {users_count[2]}
8  coverage exclude -du FSM_010 -togglenode {users_count[3]}
9  coverage exclude -du FSM_010 -togglenode {users_count[4]}
10 coverage exclude -du FSM_010 -togglenode {users_count[5]}
11 coverage exclude -du FSM_010 -togglenode {users_count[6]}
12 coverage exclude -du FSM_010 -togglenode {users_count[7]}
13 coverage exclude -du FSM_010 -togglenode {users_count[8]}
14 coverage exclude -du FSM_010 -togglenode {users_count[9]}
15 coverage exclude -src FSM_010.v -line 42 -code s
16 coverage exclude -src FSM_010.v -line 42 -code b
17 quit -sim
18 vcover report FSM_010_tb.ucdb -details -annotate -all -output coverage_rpt.txt

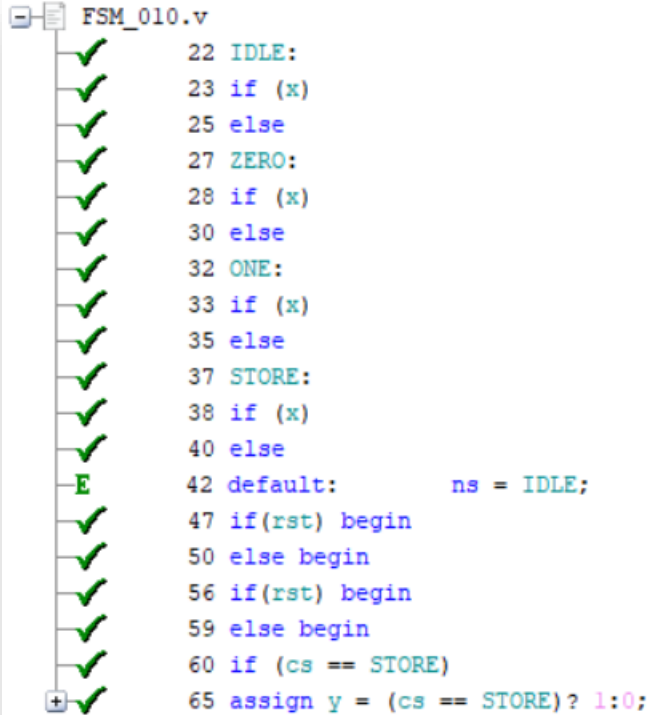
```

6. QuestaSim Waveform:



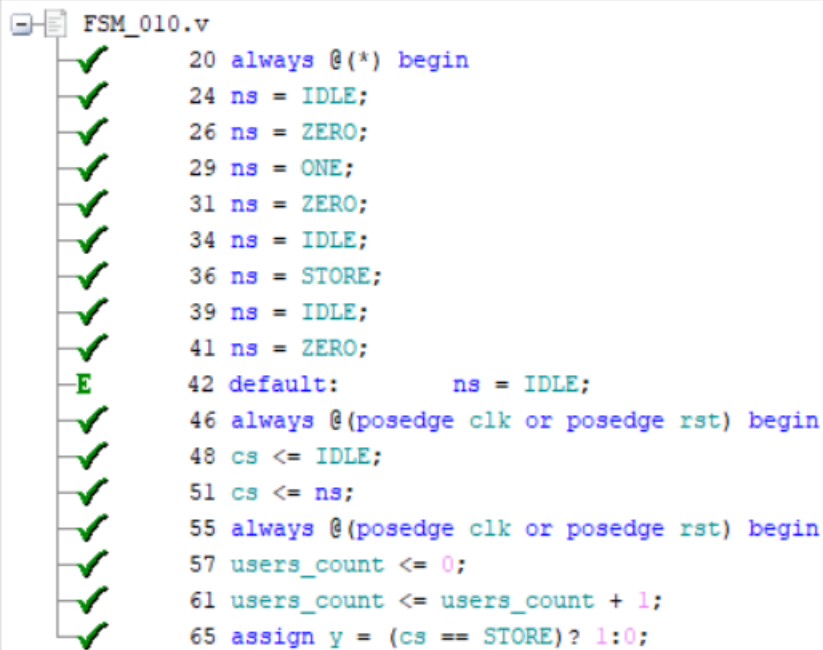
7. Code Coverage:

Branch Coverage:



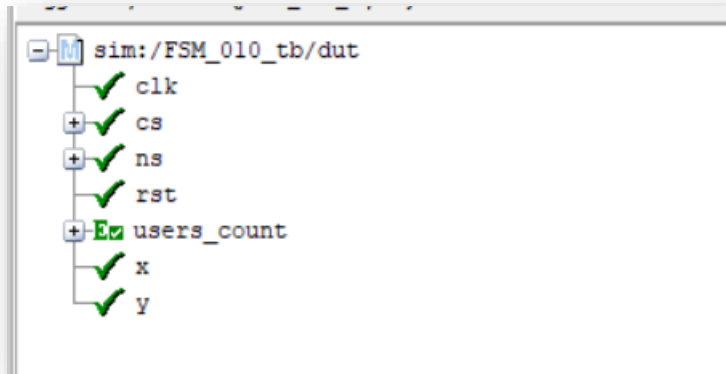
```
FSM_010.v
22 IDLE:
23 if (x)
25 else
27 ZERO:
28 if (x)
30 else
32 ONE:
33 if (x)
35 else
37 STORE:
38 if (x)
40 else
42 default:      ns = IDLE;
47 if(rst) begin
50 else begin
56 if(rst) begin
59 else begin
60 if (cs == STORE)
65 assign y = (cs == STORE)? 1:0;
```

Statement Coverage:



```
FSM_010.v
20 always @(*) begin
24 ns = IDLE;
26 ns = ZERO;
29 ns = ONE;
31 ns = ZERO;
34 ns = IDLE;
36 ns = STORE;
39 ns = IDLE;
41 ns = ZERO;
42 default:      ns = IDLE;
46 always @(posedge clk or posedge rst) begin
48 cs <= IDLE;
51 cs <= ns;
55 always @(posedge clk or posedge rst) begin
57 users_count <= 0;
61 users_count <= users_count + 1;
65 assign y = (cs == STORE)? 1:0;
```


Toggle Coverage:



Note: Users_count[9:2] are excluded to reduce the number of test

8. Coverage Report:

Branch Coverage:

```
=====
=== Instance: /\FSM_010_tb#dut
=== Design Unit: work.FSM_010
=====
Branch Coverage:
  Enabled Coverage          Bins    Hits    Misses  Coverage
  -----
  Branches                  20      20       0   100.00%
=====Branch Details=====
```

FSM Coverage:

```
FSM Coverage:
  Enabled Coverage          Bins    Hits    Misses  Coverage
  -----
  FSM States                 4       4       0   100.00%
  FSM Transitions            7       6       1    85.71%
=====FSM Details=====
```

Statement Converge:

Statement Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage

Statements	16	16	0	100.00%

Toggle & Total Coverage:

Toggle Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage

Toggles	20	20	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\FSM_010_tb#dut --

Node	1H->0L	0L->1H	"Coverage"

clk	1	1	100.00
cs[1-0]	1	1	100.00
ns[1-0]	1	1	100.00
rst	1	1	100.00
users_count[1-0]	1	1	100.00
x	1	1	100.00
y	1	1	100.00

Total Node Count = 10
Toggled Node Count = 10
Untoggled Node Count = 0

Toggle Coverage = 100.00% (20 of 20 bins)

Total Coverage By Instance (filtered view): 97.61%

Note: not 100% coverage due to FSM transitions

9. Functional Coverage:

Covergroup	Metric	Goal	Bins	Status
TYPE /FSM_010_pkg/fsm_transaction/cg_x	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
Coverpoint x_trans	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
Covergroup instance \FSM_010_pkg::fsm_transaction::cg_x	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
Coverpoint x_trans	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
bin x_0_1_0	7	1	-	Covered
TOTAL COVERGROUP COVERAGE: 100.00% COVERGROUP TYPES: 1				
Total Coverage By Instance (filtered view): 100.00%				

Q4)

Questasim Output:

```
# Number of memory elements initialized: 4
# Address 00000 : Data a50400
# Address 00400 : Data 123456
# Address 00401 : Data 789abc
# Address fffff : Data 0fle2d
```

Q5)

```
1  module struct();
2
3      typedef bit [6:0] bits_7;
4
5      typedef struct{
6          bits_7 header;
7          bits_7 cmd;
8          bits_7 data;
9          bits_7 crc;
10     } frame;
11
12
13     frame my_packet;
14
15     my_packet.header = 7'h5A;
16     my_packet.cmd = 7'h0;
17     my_packet.data = 7'h0;
18     my_packet.crc = 7'h0;
19
20 endmodule
```